

UNIVERSITÀ DEGLI STUDI DI BERGAMO

Dipartimento di Ingegneria

Corso di Laurea in Ingegneria Informatica



FIND & CHARGE

Project plan

Docente:

Prof. Gargantini Angelo

Prof.ssa. Bonfanti Silvia

Gruppo di lavoro:

Maistrello Tommaso 1093176

Mascheretti Davide 1093276

Morgera Claudio 1093069

Valenari Francesco 1092572

Anno Accademico 2025/2026

Sommario

INTRODUZIONE.....	4
MODELLO DI PROCESSO	5
Strategie utilizzate	5
Suddivisione del tempo	5
ORGANIZZAZIONE DEL PROCESSO.....	6
Ruoli.....	6
Attività	6
STANDARD, LINEE GUIDE E PROCEDURE	7
Gestione codice	7
Standard utilizzati	7
Gestione del database	7
Interfaccia grafica	7
ATTIVITÀ DI GESTIONE	9
POTENZIALI RISCHI.....	10
Rischi organizzativi e gestionali	10
Rischi tecnici	10
Rischi di sicurezza	10
Rischi legati alle funzionalità dell'app.....	10
PERSONALE.....	12
Ruoli.....	12
METODI E TECNICHE.....	13
GARANZIE DI QUALITÀ	14
PACCHETTI DI LAVORO.....	15
1- Inizializzazione progetto e configurazione dipendenze:.....	15
2- Gestione Utente:	15
3- Mappa e Visualizzazione Colonnine:	15
4- Sistema di Prenotazione:	15
5- Menu laterale e pagine dedicate:.....	15
6- Generazione e Gestione QR Code:	15
7- Ottimizzazione e test automatici:	15
8- Modifiche e aggiunte finali:.....	15

RISORSE	16
Risorse umane	16
Software (piattaforme, versioni delle dipendenze)	16
Software (comunicazione e gestione)	16
Hardware	16
BUDGET	17
Costi software	17
Costi hardware.....	17
Costi di personale	17
CAMBIAMENTI.....	18
CONSEGNA.....	19

INTRODUZIONE

Il progetto consiste nello sviluppo di una web app che permette agli utenti di prenotare colonnine di ricarica per veicoli elettrici.

L'app consente agli utenti di bloccare uno slot orario dalla durata di 30 minuti scegliendo direttamente da una mappa integrata, centrata sulla posizione corrente dell'utente. All'apertura l'utente visualizza la mappa con i marker che identificano le colonnine presenti nella zona scelta e può accedere, o registrarsi se necessario, tramite un pulsante presente nel menù. Solo dopo aver eseguito l'accesso l'utente può effettuare una prenotazione.

La prenotazione può avvenire sia tramite una barra laterale sulla mappa che appare nel momento in cui si clicca su un marker, sia tramite la pagina delle colonnine presente nel menù laterale, contenente tutto l'elenco completo delle colonnine e le loro caratteristiche. Questa permette inoltre di filtrare la ricerca per tipologia, nome e posizione.

Una volta confermata la prenotazione, essa viene salvata nella sezione "prenotazioni" dell'app. Cinque minuti prima dell'inizio dello slot orario prenotato, verrà generato un QR Code univoco della prenotazione; questo permette tramite scansione diretta sul lettore presente sulla colonnina di controllare che tutto corrisponda per poi avviare l'erogazione della corrente e la ricarica dell'auto (la gestione di questa fase non rientra nel nostro progetto). L'idea del QR code permette che solo l'utente che ha prenotato in quello slot possa usufruire del servizio, evitando abusi.

L'applicazione include inoltre funzionalità di valutazione delle colonnine e segnalazione dei guasti o occupazioni illecite.

L'utente può in ogni momento effettuare il logout dall'applicazione tramite un apposito pulsante presente nel menu.

La nostra applicazione andrebbe a risolvere il problema che, in un futuro con un maggior numero di auto elettriche, una persona che necessita ricarica non rischia di trovare la colonnina occupata. Risolve dunque il problema di assegnazione e il diritto di ricarica ma non il possibile abuso fisico della colonnina.

È inoltre molto semplice rendere questa app scalabile anche a grandi città, in cui gli utenti possono collaborare segnalando anche la presenza di nuove colonnine o condividere feedback e suggerimenti agli sviluppatori.

MODELLO DI PROCESSO

Il modello adottato è un modello di tipo agile.

Tale approccio permette di realizzare piccoli incrementi molto frequenti che fanno proseguire lo sviluppo della nostra applicazione senza avere documenti e fasi rigide da seguire, come invece avverrebbe per altri modelli. L'adozione di un metodo agile favorisce maggiore libertà, collaborazione continua e assicura l'esistenza di un software funzionante in ogni fase del progetto. Si pone dunque maggiore attenzione su design semplici, meeting frequenti ma brevi e processi di cambiamento e integrazione continua.

L'approccio si ispira in particolare al modello SCRUM, applicato però in maniera leggera, mantenendo i principi fondamentali ed implementando fasi e gestione dei ruoli in maniera meno rigida.

Strategie utilizzate

Prima dell'avvio effettivo del progetto si svolgono due settimane di prototipazione, considerabili come Sprint 0. Questo non sarebbe presente nel metodo SCRUM ufficiale ma è necessario per comprendere il funzionamento del database, dell'interfaccia grafica e delle attività base che saranno implementate.

In questo modo si ottiene una visione oggettiva degli obiettivi che possiamo raggiungere, delle difficoltà che potremmo incontrare e permette di fare una stima temporale in relazione alle funzionalità.

Questa parte di prototipazione è di tipologia sia usa e getta sia evolutiva. Il progetto verrà avviato in maniera definitiva da zero (usa e getta) e parti del codice del prototipo verranno riutilizzate (evolutiva).

È previsto l'utilizzo di una Kanban Board per tenere traccia dello sviluppo del lavoro.

Suddivisione del tempo

Planning: uno/due meeting di discussione riguardo funzioni da implementare e obiettivi da raggiungere nella fase di Sprint.

Sprint: dieci giorni lavorativi di implementazione effettiva delle funzionalità decise in precedenza nei meeting in presenza.

Review: uno/due meeting di revisione per verificare che tutte le funzionalità dell'ultimo Sprint siano corrette e funzionanti. Lo sprint retrospective è compreso in questa fase.

Daily SCRUM: ogni giorno il team si ritrova fisicamente per una condivisione delle modifiche apportate e degli eventuali problemi riscontrati.

La scelta di questo approccio permette quindi di gestire in maniera più flessibile le priorità, la comunicazione e gli aggiornamenti del software.

A seguire un esempio di pianificazione per i primi due sprint:

Fase	Durata
Sprint 0 – Prototipazione iniziale	2 settimane
Sprint 1 – Planning	1/2 meeting
Sprint 1 – Sprint	10 giorni
Sprint 1 – Review	1/2 meeting
Sprint 2 – Planning	1/2 meeting
Sprint 2 – Sprint	10 giorni
Sprint 2 – Review	1/2 meeting

Il numero di sprint da effettuare sarà commisurato alle funzionalità che il gruppo implementerà.

ORGANIZZAZIONE DEL PROCESSO

Sono stati definiti i seguenti ruoli, in accordo con le regole del metodo ispirato a SCRUM adottato dal gruppo.
Product owner - Davide Mascheretti: responsabile della gestione generale del progetto, delle risorse e delle priorità.

SCRUM master - Tommaso Maistrello: responsabile del corretto svolgimento dell'approccio agile, della comunicazione nel team e risolutore di problemi di natura tecnica e non.

Team di implementazione - Tommaso Maistrello, Davide Mascheretti, Claudio Morgera, Francesco Valenari: responsabili dello sviluppo del software, della documentazione e del testing.

Dato il team ridotto il Product Owner e lo Scrum Master svolgono anche il ruolo di sviluppatori.

Ruoli

Inoltre, seppur non sia coerente totalmente con il metodo SCRUM classico, all'interno del team di implementazione abbiamo suddiviso approssimativamente i ruoli in base alle competenze prevalenti.

Frontend: sviluppo interfaccia, mappa interattiva, passaggio tra pagine, menù.

Backend + Database: gestione Firebase Realtime Database, controlli utente e prenotazioni, salvataggio dati, sicurezza.

Testing: controllo corretta funzionalità di metodi e di classi, verifica eccezioni.

Documentazione: project plan, diagrammi UML, informazioni generali.

Le specifiche e l'assegnazione di questi ruoli all'interno del team sono illustrate in maniera più dettagliata all'interno del paragrafo "Personale". Nonostante la divisione dei ruoli, nel caso di necessità un membro del gruppo può contribuire alla collaborazione con il membro affidato a quel ruolo.

La conoscenza deve essere utilizzata in maniera trasversale e il gruppo deve adottare un approccio collaborativo.

La documentazione chiara e dettagliata è considerata responsabilità condivisa tra tutti i membri del gruppo, essa deve essere il compito di tutti.

Questa tipologia di organizzazione permette ai membri del gruppo di avere una definita suddivisione dei ruoli, in cui ognuno è a conoscenza delle sue responsabilità al fine di avere una corretta evoluzione del progetto.

Attività

La comunicazione quotidiana interna avviene tramite un gruppo Whatsapp dedicato, utilizzato per scambio di informazioni, aggiornamenti, domande inerenti al progetto o a problemi riscontrati.

Sono previsti inoltre brevi meeting principalmente in presenza per risolvere problemi e per eventuali chiarimenti.

Decisioni, urgenze e task vengono gestiti dal gruppo stesso durante i meeting settimanali di durata superiore rispetto a quelli quotidiani, in cui si decide se sono necessari cambiamenti nei metodi di lavoro del team.

La gestione delle attività che ogni membro svolge sul codice viene gestita tramite l'utilizzo di GitHub privato condiviso con il gruppo. Esso è inoltre utilizzato per la tracciabilità e per la gestione delle issues.

La documentazione viene inizialmente caricata e modificata in un file condiviso in Google Drive per poi essere integrata ufficialmente nella repository.

È necessario lavoro costante da parte di tutto il team.

STANDARD, LINEE GUIDE E PROCEDURE

Gestione codice

Per gestire tutte le procedure e tutti i cambiamenti del nostro progetto è obbligatorio utilizzare GitHub, gestore di configurazione online distribuito che permette ad ognuno di avere una propria local repository.

Riguardo la documentazione è necessaria chiarezza sia per le classi aggiunte, che devono essere esplicitamente e dettagliatamente spiegate, sia per tutti i commit che vengono fatti.

La tracciabilità delle modifiche deve essere un aspetto fondamentale.

Nel titolo del commit ci devono essere in generale le modifiche principali apportate e cosa cambia di importante rispetto alla versione precedente. Nella descrizione è possibile aggiungere tutte le informazioni necessarie che descrivono i cambiamenti passo per passo, le scelte effettuate, le motivazioni (se necessarie) e tutto ciò che permette di comprendere al meglio anche ad una persona esterna cosa è cambiato in seguito alle modifiche effettuate.

È obbligatorio effettuare commit frequenti, anche se piccoli, se rappresentano una modifica logica.

Nel caso di modifiche con bug presenti (leggeri, che non annullano il funzionamento generale) è consigliato comunque fare commit, magari aggiungendo la descrizione dell'errore o un issue. In questo modo ognuno è aggiornato alla versione corretta ed è possibile risolvere il bug o in parallelo allo sviluppo o alla fine del progetto.

Ci si aspetta che ognuno rispetti le linee guida in maniera autonoma, nessun membro del gruppo verifica che gli altri membri rispettino le linee guida per modifiche minori. Nonostante ciò il gruppo adotta un approccio di revisione collettiva in caso di funzionalità maggiori implementate. Ogni membro del gruppo è responsabile della qualità e della coerenza del codice che scrive.

Il gruppo utilizza inoltre la tabella presente nel documento di specifica di requisiti per visualizzare lo stato di avanzamento del progetto, segnando la data di raggiungimento di determinati requisiti.

Standard utilizzati

Gli standard di programmazione sono quelli classici di **Java** definiti da Oracle. Vengono rispettate convenzioni su classi, attributi e metodi.

Ogni classe deve essere introdotta da una descrizione generale del suo funzionamento, degli autori e della versione che abbiamo modificato.

Per ogni metodo invece è necessaria una descrizione generica e una spiegazione dettagliata dei parametri utilizzati e del tipo di ritorno della funzione.

Queste funzioni avvengono tramite Javadoc (/**). Commenti secondari utili per la comprensione devono essere inseriti all'interno del metodo stesso, nel modo `//` o `/*`.

Gestione del database

Per la gestione dei dati si utilizza **Firebase Realtime Database**, basato su linguaggio NoSQL. Questo permette l'accesso simultaneo di più utenti e l'aggiornamento in tempo reale. Si utilizzano regole standard (preimpostate da Firebase), database privato, in modo che solo i membri interni al gruppo possano visualizzare e modificare. Il ruolo assegnato al service account di Firebase è quello minimo, utile solo per leggere e scrivere sul database.

Tramite questo si gestisce tutta la parte di salvataggio, autenticazione, controlli, password criptate.

Si prevedono cinque nodi principali: utenti, prenotazioni, colonnine, auto, recensioni e segnalazioni.

La gestione dei sotto-nodi sarà definita in maniera più chiara in fase di implementazione.

Interfaccia grafica

Per la parte grafica si utilizza **Vaadin**.

Oltre a gestire la parte visiva delle interfacce (già integrata), rende più semplice il collegamento con la parte back-end dell'applicazione.

Per la gestione stili e colore di caratteri e in generale della parte web visiva abbiamo integrato Vaadin (che è limitato sotto tale punto di vista) con delle aggiunte di codice **CSS** e **JavaScript**.

Per la mappa si è optato per **Leaflet**, un software open source che ci permette di usufruire di mappe interattive senza troppi problemi (inizialmente si era pensato alle API di Google Maps ma la procedura di ottenimento della chiave era molto più complessa).

Queste aggiunte rendono la nostra applicazione più professionale e, in un futuro, realmente utilizzabile. È infatti facilmente scalabile: si possono aggiungere nuovi nodi e funzionalità mantenendo il design implementato dal team.

ATTIVITÀ DI GESTIONE

La gestione del progetto avviene attraverso incontri in presenza soprattutto in ambiente universitario, che avvengono in particolare nei giorni adibiti alla fase di planning e di sprint review, oltre al daily SCRUM.

Durante l'incontro ogni membro aggiorna il gruppo sulle modifiche apportate al codice, eventuali problemi che ha rilevato, accorgimenti e un'ipotetica suddivisione dei compiti che devono essere assegnati agli altri membri nel caso siano necessari per lo sviluppo del progetto.

Naturalmente tutto ciò, oltre ad essere tema di incontro, deve essere dettagliatamente descritto nella documentazione o negli issue tramite GitHub.

Eventuali dubbi e domande sorti dopo i vari incontri possono essere chiariti anche tramite gruppo Whatsapp creato per una messaggistica rapida, a cui può seguire una chiamata o una discussione per la ricerca di una possibile risoluzione della problematica.

La gestione degli errori e la ricerca della soluzione è guidata dallo SCRUM Master del team ed è poi implementata dal team di sviluppatori in base al ruolo assegnato.

In caso di mancata soluzione al problema, e la conseguente impossibilità di chiusura della issue nel caso sia stata creata, si opterà per un nuovo incontro di gruppo.

POTENZIALI RISCHI

Rischi organizzativi e gestionali

I principali problemi potrebbero riguardare ritardi nella stesura dei diversi diagrammi e nella consegna tendenzialmente giornaliera degli aggiornamenti e delle modifiche.

Potrebbe succedere che per alcuni periodi (di esami) sia più complicato lavorare al progetto, rallentando la sua evoluzione e diminuendo la produttività e il numero di commit effettuati.

Rischi tecnici

Da non sottovalutare sono i rischi tecnici legati alle scelte intraprese per il progetto.

Altri problemi potrebbero ad esempio riguardare il database. Utilizzando Firebase database (nessun membro del gruppo lo aveva mai utilizzato in precedenza) potrebbero sorgere problematiche di gestione dei dati, creazione database e interrogazioni. Utilizzando inoltre NoSQL i problemi potrebbero essere maggiori e più complicati da risolvere nel tempo.

È ancora più necessario in questa parte il contributo di tutto il gruppo.

Nello stesso modo vale per Vaadin. È necessaria una fase iniziale di comprensione su come utilizzarlo, cosa permette di fare e se è effettivamente utile e necessario al nostro progetto.

Un ulteriore problema si potrebbe riscontrare nell'utilizzo di Papyrus perché è complesso il lavoro simultaneo anche su diagrammi differenti.

Rischi di sicurezza

Importante è anche capire come gestire la chiave privata del service account e tutti i rischi di sicurezza ad essa legata. Infatti è altamente sconsigliato inserire la chiave direttamente sulla repository online GitHub (se privata è possibile, ma non una buona norma).

Rischi legati alle funzionalità dell'app

In generale, per avere maggiore realismo e serietà dell'app ci sono innumerevoli rischi e problemi a cui andremo incontro, vista l'assente conoscenza nell'ambito di tutti i membri del gruppo. Sono sempre possibili complessi bug che fanno parte di ogni progetto ed è necessario risolverli prima della consegna dell'elaborato. Riferendosi invece alla funzionalità dell'app i problemi riguardano principalmente la concorrenza e il fatto che più utenti possono accedere al database in maniera simultanea. Occorre implementare meccanismi di controllo che garantiscano che due utenti non possano prenotare la stessa colonnina nella stessa fascia oraria. Questo è fondamentale che venga rispettato.

I potenziali rischi che verranno rilevati con l'avanzamento dell'applicazione saranno gestiti in maniera singola, analizzando il problema e trovando la soluzione più adatta.

Qui una tabella riassuntiva:

Tipologia Rischio	Descrizione	Soluzione
Gestionale	Calo produttività e ritardi causa esami.	Pianificare durante la sessione di esame degli sprint più brevi e leggeri, senza implementare funzionalità troppo complesse.
Tecnico	Inesperienza Firebase e NoSQL	Cercare informazioni online prima di iniziare con l'implementazione vera e propria.

Tipologia Rischio	Descrizione	Soluzione
Tecnico	Inesperienza Vaadin e Leaflet	Ricerche online e collaborazione tra membri per velocizzare il processo.
Sicurezza	Esposizione della chiave privata del Service Account	Mettere la repository privata e ridurre al minimo i ruoli del service account (solo lettura e scrittura, senza accesso ad altre funzionalità). Per maggiore sicurezza mettere la chiave nel .gitignore o utilizzare variabili d'ambiente.
Funzionale	Concorrenza di prenotazioni	Gestire tramite Firebase un controllo delle prenotazioni al momento del click. Se necessario implementare le Transactions.
Qualità	Bug ed errori in corso d'opera	Revisione attenta del codice prima di caricare nella repository. Se si utilizza un branch secondario, creare pull request e, se sono modifiche importanti, attendere l'accettazione da parte di un altro membro.

PERSONALE

Il progetto prevede il lavoro e la collaborazione costante di quattro membri, studenti del corso di laurea in Ingegneria Informatica. Tutti possiedono conoscenze e competenze nello sviluppo software in Java e nell'utilizzo dell'ambiente di Eclipse.

Ciascun componente del gruppo partecipa sia alla fase di stesura di documenti sia alla fase di implementazione di codice, con attività sia di Front-End sia di Back-End, secondo le proprie competenze e i ruoli assegnati dal Product Owner.

Ruoli

Davide Mascheretti — Product Owner e Sviluppatore Back-End

Coordina le attività, pianifica le scadenze, gestisce le funzionalità dell'applicazione e verifica richieste di modifica.

Implementa la logica applicativa, la gestione del database e delle dipendenze necessarie per il corretto funzionamento dell'applicazione.

Tommaso Maistrello — SCRUM master e Sviluppatore Front-End

Cura la documentazione e raccoglie i requisiti funzionali e non funzionali in un file specifico in collaborazione con il Product Owner.

Realizza l'interfaccia utente.

Francesco Valenari — Sviluppatore Front-End e progettista software

Definisce l'architettura del sistema, la struttura del codice e i diagrammi UML, garantendo coerenza e qualità del design.

Aiuta nella realizzazione dell'interfaccia utente.

Claudio Morgera — Sviluppatore Back-End e testing

Aiuta nell'implementazione della logica applicativa e gestisce l'integrazione tra moduli, servizi e database.

L'impegno dei membri sarà costante per tutta la durata del progetto, con maggiore intensità nelle fasi di sviluppo e collaudo. Si sottolinea inoltre come la documentazione e la fase di testing sono compiti di ogni membro del team, in particolare dopo che effettua modifiche al codice. Il confine tra questi ruoli è labile.

METODI E TECNICHE

Durante lo sviluppo del progetto saranno adottati metodi e tecniche di Ingegneria del Software che coprono tutte le fasi del ciclo di vita.

Ingegneria dei requisiti: i requisiti saranno raccolti tramite incontri di gruppo e discussioni guidate dal Product Owner e dallo SCRUM Master. Saranno documentati in modo strutturato utilizzando diagrammi dei casi d'uso e specifiche testuali. Verrà poi redatto un documento a parte denominato "Specifica dei requisiti" che raggruppa e suddivide i requisiti in base alla loro funzionalità e prioritizzazione.

È stata infatti utilizzata una prioritizzazione di MoSCoW: la suddivisione dei requisiti in must, should, could e won't permette di definire in maniera più chiara le scadenze e il tempo da dedicare ad ogni requisito, dai più importanti e centrali a quelli minori.

Progettazione: sarà impiegato un approccio Object-Oriented (OO), con l'ausilio di diagrammi UML per rappresentare classi, sequenze e componenti. La progettazione sarà validata in gruppo prima dell'inizio della codifica.

Implementazione: lo sviluppo sarà realizzato in Java, utilizzando l'ambiente Eclipse IDE.

Verrà seguito un processo incrementale e iterativo dove ciascun componente sarà sviluppato, testato e integrato progressivamente per tutta la durata del progetto.

Controllo di versione: il codice sarà gestito tramite Git, con repository su GitHub per il versionamento, la collaborazione e la revisione del codice. Ogni membro effettuerà commit regolari e aprirà pull request per l'integrazione nel caso di lavoro su branch secondari.

Documentazione tecnica: sarà prodotta e mantenuta durante tutto il progetto, comprendendo specifiche dei requisiti, manuali tecnici e relazioni di test. I documenti saranno salvati in formato digitale condiviso e versionati insieme al codice.

Ambiente di prova: i test saranno condotti sui computer personali dotati di Java Development Kit (JDK) ed Eclipse, replicando l'ambiente di esecuzione previsto.

Integrazione e test: i componenti saranno testati manualmente ad ogni commit mentre il testing automatico verrà effettuato durante la fase di implementazione. Sono previsti inoltre test automatici con appositi strumenti prima della consegna (vedi "Pacchetti di lavoro - 7").

GARANZIE DI QUALITÀ

La garanzia della qualità del software sarà assicurata attraverso un insieme di procedure e controlli mirati a garantire che il prodotto finale soddisfi i requisiti definiti.

Revisione dei requisiti e del design: ogni documento e modello progettuale sarà rivisto in gruppo per verificarne la completezza e la coerenza con gli obiettivi del progetto.

Controllo della qualità del codice: verranno adottate convenzioni di programmazione condivise (stile, nomenclatura, formattazione) e saranno effettuate revisioni periodiche del codice collaborative da parte degli altri membri del team, che forniranno consigli e proporranno modifiche.

Test di qualità: saranno realizzati test unitari, di integrazione e di sistema per garantire il corretto funzionamento del software. Ogni membro del team è responsabile di fare test sulle parti di codice che implementa. Se il codice non è funzionante deve segnalarlo a tutto il team.

La priorità è su test manuali e funzionali effettuati su PC personali.

Verranno effettuati test automatici con JUnit per testare principalmente l'inserimento dati da parte dell'utente.

Tracciabilità: verrà mantenuta una corrispondenza tra requisiti, componenti implementati e casi di test, per assicurare la copertura completa delle funzionalità. Per la parte di requisiti verrà inoltre tracciata la data del raggiungimento del requisito nell'apposito file. Ogni incontro, modifica e problema verranno tracciati all'interno di appositi documenti o tramite le issues di GitHub.

Gestione delle modifiche: eventuali correzioni o miglioramenti saranno tracciati nel sistema GitHub, garantendo il controllo delle revisioni e la riproducibilità del lavoro.

Verifica finale: prima della consegna, il software sarà sottoposto a un test di accettazione finale per confermare la conformità alle specifiche e la stabilità del sistema. Verrà chiesto ad ogni membro del team di testare più volte il software al fine di trovare problematiche non notate in precedenza.

PACCHETTI DI LAVORO

Il progetto è suddiviso nei seguenti macro-blocchi, che verranno poi scomposti tra i diversi membri del gruppo, in relazione al ruolo a loro assegnato.

1- Inizializzazione progetto e configurazione dipendenze:

- Creazione repository privata GitHub e struttura progetto Vaadin e Spring Boot con dipendenze nel pom
- Configurazione Firebase (Realtime Database) e chiavi di accesso nella classe apposita
- Creazione prima classe Application per verificare funzionamento

2- Gestione Utente:

- Implementazione viste di registrazione e login
- Creazione collegamento tra la pagina principale e la pagina di login
- Implementazione del database nelle pagine, salvataggio utenti e controllo username già esistente
- Controlli su campi vuoti, mail non valide, password verificate (inserimento doppio della password per verifica)

3- Mappa e Visualizzazione Colonnine:

- Implementazione delle dipendenze necessarie per utilizzare Leaflet
- Implementazione della pagina principale con mappa e menu laterale
- Caricamento dei marker delle colonnine da Firebase sulla mappa Leaflet, con coordinate
- Implementazione barra di ricerca e filtri (tipo, nome, posizione) per ricerca colonnine
- Visualizzazione sidebar con dettagli colonnina al click sul marker con al suo interno tasto prenota, recensioni e segnalazioni
- Aggiunta barra laterale e tasto logout

4- Sistema di Prenotazione:

- Sviluppo interfaccia di selezione data e slot orari con menù a tendina nella sidebar che si visualizza dopo il click sulla colonnina
- Implementazione logica backend per la validazione della prenotazione, gestione DB con salvataggio delle prenotazioni e controlli
- Gestione della concorrenza con controllo nel database di un nodo uguale (colonnina/data/orario)
- Implementazione in Firebase Service di tutti i nodi necessari per corretta gestione DB

5- Menu laterale e pagine dedicate:

- Implementazione delle pagine dedicate (utente, colonnine, prenotazioni, mappa) nella barra laterale
- Implementazione parte grafica in queste pagine
- Implementazione collegamento con Database nella pagina utente per visualizzare tutte le prenotazioni

6- Generazione e Gestione QR Code:

- Implementazione dipendenza per la generazione del QR Code univoco basato sulla stringa assegnata alla prenotazione
- Salvataggio del QR code nel database, utilizzabile poi nella pagina utente

7- Ottimizzazione e test finali:

- Testing automatici finali
- Aggiunte/modifiche JS e CSS per interfaccia e parte grafica

8- Modifiche e aggiunte finali:

- Implementazione di funzionalità extra (notifiche, segnalazioni...)
- Modifiche grafiche per interfaccia più user-friendly

RISORSE

Risorse umane

- Quattro membri del team, ciascuno con il proprio ruolo e responsabilità (descritti meglio nella sezione “Personale”)

Software (piattaforme, versioni delle dipendenze)

- IDE: Eclipse IDE for Java Developers - 2025-09
- Papyrus - versione 2024-06
- JDK (Java Development Kit) - versione 17
- GitHub (per repository privato)
- Firebase Realtime Database (piano gratuito Spark) - versione 9.2.0
- Vaadin - versione 24.3.3
- Spring Boot - versione 3.2.2
- Leaflet - versione 4.1.1
- Apexcharts - versione 23.0.1
- BCrypt per criptare le password
- ZXing per la generazione di QR code – versione 3.5.2
- Nessuna licenza a pagamento necessaria, tutti i piani sono gratuiti e limitati

Software (comunicazione e gestione)

- WhatsApp (comunicazione quotidiana)
- Google Drive (documentazione iniziale aggiornata)
- GitHub per la revisione dei commit, la tracciabilità delle modifiche e la gestione delle issues
- GitHub Desktop per i commit e le modifiche

Hardware

- PC di ogni membro del team già in suo possesso

BUDGET

Trattandosi di un progetto sviluppato in ambito accademico, il budget monetario è pari a zero.

Costi software

Nulli: tutte le tecnologie e piattaforme utilizzate (Eclipse, Papyrus, Vaadin, Leaflet, Firebase piano base, GitHub Free) sono gratuite per gli scopi del progetto.

Costi hardware

Nulli: ogni membro utilizza il proprio PC personale già in suo possesso.

Costi di personale

Nulli: Il lavoro fa parte del percorso formativo universitario. Non sono previsti compensi economici.

Sebbene non ci siano costi monetari, si stima un costo temporale di circa 30-40 ore-uomo a testa per un totale di 120-160 ore di sviluppo puro (la parte di studio, comprensione e comunicazione nel team non sono comprese).

CAMBIAMENTI

Nel corso del progetto potranno verificarsi modifiche ai requisiti o all'implementazione di grande impatto sul progetto. È quindi necessario gestire tali cambiamenti in modo ordinato e tracciabile, per evitare incoerenze o perdite di qualità del progetto.

Ogni modifica effettuata deve essere coerente, completa, tracciata e non ambigua.

Poiché il progetto segue un approccio iterativo e leggero, i cambiamenti corposi saranno gestiti in maniera agile. Nonostante ciò, è essenziale che in ogni momento il software sia correttamente funzionante.

Tuttavia, per garantire il controllo e la coerenza del lavoro svolto, verranno seguite procedure condivise: ogni proposta di modifica di rilevante importanza (ai requisiti, al design o al codice) dovrà essere discussa e approvata dal gruppo, in particolare dal Product Owner.

Una volta approvata, la modifica sarà documentata indicando la motivazione, l'impatto stimato e i file coinvolti.

In caso di modifiche minori ciascun membro del gruppo si assume la responsabilità delle proprie implementazioni e non è necessaria un'approvazione da parte del team.

Per evitare ambiguità si definiscono "modifiche di rilevante importanza":

1. modifiche alla struttura del database (aggiunta di nodi o modifiche di Firebase);
2. aggiunta di funzionalità che prima non esistevano;
3. aggiunta di nuove classi o di metodi rilevanti a classi preesistenti.

Le modifiche al codice saranno gestite tramite GitHub, creando un nuovo branch o commit dedicato e una pull request per la revisione se necessari.

Nonostante sia utilizzato un metodo agile è importante che la descrizione del commit sia ben dettagliata e spieghi in maniera precisa i cambiamenti rispetto alla versione precedente.

Nel caso di branch secondario, a seguito di pull request ci si aspetta che un altro membro del gruppo visualizzi la richiesta e la approvi o non la approvi con le adeguate motivazioni.

Le modifiche che comportano aggiornamenti alla documentazione tecnica (ad esempio i diagrammi UML) saranno riportate anche nei relativi file, per mantenere la coerenza tra codice e documentazione.

Questo approccio permette di mantenere un controllo chiaro sulle versioni del software e di prevenire modifiche non tracciate che potrebbero compromettere la qualità e aumentare i tempi di sviluppo.

CONSEGNA

Per la consegna del progetto è obbligatorio mettere il tutto in un repository GitHub da condividere con il professore (account GitHub: garganti) e con l'esercitatrice (account GitHub: silviabonfanti).

I tempi di consegna possono variare in base alla decisione del gruppo nel voler svolgere l'esame di Ingegneria del Software.

Il project plan deve essere pronto un mese circa prima dell'esame e il progetto deve essere completato cinque giorni prima della sua presentazione.